

RECON 6

This is a Python translation and upgrade of RECON5-5, a Fortran program (©Sukumar & Breneman, 2001) that computes **TAE** (Transferable Atom Equivalent) molecular descriptors — quantum-chemistry-derived property descriptors assembled from a precomputed atom-type database, without running a new quantum calculation for each molecule.

Transferable Atom Equivalent Reconstruction (**TAE-RECON**) is a computational chemistry method that rapidly generates highly accurate quantum mechanical molecular descriptors. Originally developed by the late Prof. Curt Breneman's group at RPI, it drastically accelerates molecular modeling by assembling pre-computed atomic electron-density fragments, skipping time-consuming *ab initio* calculations.

The core of the algorithm is the Atomtyper and TAE descriptor pipeline: reading a molecule, classifying each atom into a 49-character alphanumeric code, based on its atomic number, number of bonded neighbours, atomic numbers of bonded neighbours, and size of ring, if any, matching that code against the TAE database, and combining the matched atomic descriptors into whole-molecule descriptors. This includes several features not present in earlier versions, and has been prepared with the help of Claude (Sonnet 4.6)

How TAE-RECON Works

- **Fragment-Based:** It uses Bader's Quantum Theory of Atoms in Molecules (**AIM**) to create a library of atomic charge density fragments, whose properties are computed at quantum mechanical accuracy.
- **Reconstruction:** When given a large molecule, or a large database of molecules, the **RECON** program matches atom types, retrieves the appropriate precomputed density fragments and stitches them together to reconstruct the molecular electron density and molecular properties.
- **Efficiency:** It calculates physical properties (like electrostatic potentials, local average ionization energies, Fukui functions, electron density Laplacians, etc) for tens of thousands of molecules in seconds.

Key Applications

- **QSAR/QSPR Modeling:** TAE-RECON has been used extensively in drug and materials discovery and toxicology to build highly predictive models for molecular properties, and for protein separations.
- **Database Mining:** Rapidly screens large chemical databases or virtual compounds for promising drug candidates.

Supported input formats

- SDF (MDL molfile, V2000)
- MOL2 (Tripos SYBYL)
- PDB (via CONECT records, or distance-based connectivity using a bond-length table)
- Gaussian`.com`.gjf` input, Cartesian coordinates only (no z-matrix/internal-coordinate support). Connectivity is always distance-based, the same as a CONECT-less PDB file, since the format has no bond list at all.

- ORCA input (.inp`/orca`), Cartesian `\* xyz <charge> <multiplicity>` coordinate block only (other ORCA input styles, e.g. internal coordinates, are not supported). Everything before the `\* xyz` marker (route lines, `%block ... end` sections) is skipped automatically. Connectivity is distance-based, same as Gaussian/PDB.
- SMILES (a simplified parser matching the capabilities of the original

`rsmiles.f` — see **Known limitations** below)

Installation

```
```bash
```

```
pip install -e .
```

```
```
```

Requires Python 3.8+. No third-party dependencies — the standard library is sufficient.

Usage

- **Command line** (recommended)

```
```bash (or Windows Terminal)
```

```
python -m recon6 --data-dir /path/to/DATA molecule.sdf -o results.csv
```

```
```
```

```
```bash (or Windows Terminal)
```

```
python -m recon6 --data-dir /path/to/DATA --fmt smiles smiles_list.txt -o results.csv
```

```
```
```

Other optional flags:

--gnn-output # Outputs atom, bond and graph attributes for a Graph Neural Network;

--no-return # Writes output a single molecule at a time (useful for large databases);

--log-file # Produces silent run and logs messages instead (useful for large databases)

--error-log # error logging of any missing atom types or poorly typed atoms. This is important information for the user, as it relates to the quality of atom typing and potential accuracy of ML models developed using these descriptors for a given dataset.

The bond-length table is read automatically from ``<data-dir>/bond``; pass ``--bond-file`` only if you need to override that location.

Run `python -m recon6 --help`` for all options.

- **As a library**

```
```python
```

```
from recon6.recon import ReconConfig, run_recon
config = ReconConfig(
 data_dir="/path/to/DATA", # directory of TAE .dat files
 input_files=["molecule.sdf"],
```

```

fmt="sdf", # or "mol2", "pdb", "gaussian", "orca", "smiles", "auto"
output_csv="results.csv", # optional
output_gnn="molecules.jsonl", # optional GNN export (JSONL or directory of .json files)
error_log="recon.err", # optional atom-typing quality report (CSV)
bond_file defaults to '<data_dir>/bond'; pass an explicit path here only if your bond-length
table lives somewhere else.
return_results=True, # set False for large batches (returns []) but still
 # stream-writes CSV/GNN/error output with bounded memory)
log_file=None, # set to a path to redirect all notes/warnings to file
)
results = run_recon(config)
for r in results:
 print(r["Molecule"], r["Energy"], r["chi"])
` ``

```

Each entry in `results` is a dict of descriptor name -> value (Energy, Population, VOLTAE, SurfArea, SIK, SIG, SIEP, Fuk, Lapl, chi, and the associated min/max/bin-fraction variants). The default CLI output filename is `recon.csv`.

### Large-batch usage (memory and performance)

By default `run\_recon` accumulates all results in a Python list and writes the CSV only after every molecule has been processed, which can exhaust memory on large datasets (observed: ~9.5 GB for 51k molecules) and produces no output until the very end. Two config flags change this:

- \*\*`return\_results=False`\*\* — disables in-memory accumulation entirely.

`run\_recon` returns `[]` but the CSV and/or GNN output are stream-written (flushed to disk after each molecule), so output appears incrementally and peak memory is bounded to roughly one molecule at a time.

- \*\*`log\_file="run.log"`\*\* — redirects all notes, TAE warnings, and per-molecule progress output from stderr to a file for the duration of the run. Useful for unattended or overnight batch jobs where stderr would otherwise go unread.

### Recommended invocation for datasets >10k molecules

```

` `` bash

python -m recon6 --data-dir /path/to/DATA --no-return --log-file run.log \
 large_dataset.sdf -o results.csv
` ``

```

The CSV header is written immediately when the output file is opened (not after the first molecule), so the file is always well-formed even if the run is interrupted.

### Atom-typing quality report (`--error-log`)

The TAE database covers a finite set of chemical environments. When an atom's computed 49-character type code doesn't match anything in the database exactly, `gettae` falls back to the closest available entry at a lower match level. RECON6 fully implements this as a structured, streaming CSV report.

Set ``ReconConfig(error_log="recon.err")`` or pass ``--error-log recon.err`` on the CLI to get one row per atom with an imperfect match, with columns:

Column	Content
<code>`Molecule`</code>	molecule name/ID
<code>`AtomIndex`</code>	1-based atom index within the molecule
<code>`Element`</code>	element symbol
<code>`AtomTypeCode`</code>	the 49-character atom-type code computed by RECON
<code>`MatchLevel`</code>	integer match quality from <code>`gettae`</code> (see below)
<code>`BestTAEEntry`</code>	the closest matching TAE database entry name
<code>`MatchQuality`</code>	human-readable label for easy filtering

**\*\*MatchLevel / MatchQuality values:\*\***

MatchLevel	MatchQuality	Meaning
3	*(not logged)*	Perfect match — all fields agree
2	<code>`near`</code>	Near match — last 2 characters differ
1	<code>`good`</code>	Good match — last 6 characters differ
0	<code>`partial`</code>	Partial match — only element + ring type matched
-1	<code>`poor`</code>	Poor match — only element matched
-2	<code>`very_poor`</code>	Very poor match — only first character matched
-3	<code>`no_match`</code>	No usable match — fallback entry used

Only atoms with `MatchLevel < 3` appear in the file. A completion summary is printed to `stderr` (or ``--log-file``) giving the total atom and molecule counts with imperfect matches. The file is flushed after each molecule, so it's readable during long runs.

**Why this matters for ML:** descriptors derived from a poor or no-match atom are computed from a structurally dissimilar reference atom type, making them unreliable. For a dataset where many atoms fall below `MatchLevel 2`, the resulting descriptors may not faithfully represent the chemistry, and any ML model built on them will have an additional source of noise or bias. Use this report to identify structural classes that are under-represented in the TAE database before committing to a modelling effort.

### Recommended for any ML-focused run

```
```bash
python -m recon6 --data-dir /path/to/DATA molecules.sdf \
  -o results.csv --error-log recon.err
```
```

On the 278-molecule tox test dataset: 71 atoms in 39 molecules had imperfect matches (mostly quaternary/tert-butyl carbons at `MatchLevel 0`partial``, and 5-membered-ring atoms at `MatchLevel 2`near``) — exactly the structural classes where the TAE database is known to be sparse.

### GNN export

``output_gnn`` (library) / ``--gnn-output`` (CLI) exports the per-atom and per-bond QMF data in a format directly usable by graph neural network pipelines, alongside the whole-molecule descriptor CSV.

## Output format

Each exported molecule is a JSON object with four keys:

```
```json
{
  "molecule_id": "3-hydroxybenzaldehyde",
  "atoms": [
    {
      "Atom": "C", "x": 1.391, "y": 0.0, "z": 0.0, "AtNum": 6,
      "Energy": -37.69, "Population": 6.12, "SurfArea": 14.3,
      "SIDel(Rho)N": 0.041, "EP1": -0.02,
      "... 162 more per-atom TAE descriptor fields ..."
    },
    "... one entry per atom ..."
  ],
  "bonds": [
    {"source": 0, "dest": 1, "bond_order": 2},
    {"source": 1, "dest": 0, "bond_order": 2},
    "... two directed entries per bond (i->j and j->i) ..."
  ],
  "molecule_descriptors": {
    "Molecule": "3-hydroxybenzaldehyde",
    "Energy": -382.36, "Population": 56.1, "chi": 2.31,
    "... full 249-column whole-molecule TAE descriptor set ..."
  }
}
```
```

The 167 per-atom descriptor field names (``NODE_FEATURE_COLS`` in ``recon6/gnn_export.py``) match the canonical column names from the Fortran QMF output format exactly, verified against real QMF files.

Atom indices in the bond list are **\*\*0-based\*\***, following the GNN convention.

## JSONL vs per-molecule JSON

Path ending in ``.jsonl`` (recommended for any batch run):

: One JSON object per line, flushed after each molecule. Readable line-by-line without loading the whole file. Safe to resume a partial run (append mode).

Any other path (treated as a directory):

: One ``<molecule_id>.json`` file per molecule, written into the directory.

## PyTorch Geometric helper

```
```python
```

```
from recon6.gnn_export import mol_to_gnn_dict, to_pyg_data
# mol and desc come from run_recon internals or your own pipeline
gnn_dict = mol_to_gnn_dict(mol, desc)
```

```

data = to_pyg_data(gnn_dict) # returns torch_geometric.data.Data
# data.x      -> node feature matrix [num_atoms, 167]
# data.edge_index -> [2, 2*num_bonds]
# data.edge_attr -> bond orders [2*num_bonds, 1]
# data.pos    -> 3-D coordinates [num_atoms, 3]
` ``

```

PyTorch and PyTorch Geometric must be installed separately — RECON6 itself does not depend on either.

Distance-based connectivity and bond-order inference

PDB, Gaussian, and ORCA input share a common connectivity module (``recon6/connectivity.py``) for the case where no explicit bond list is available: bonds are inferred from atom-pair distances against a reference bond-length table, then a bond-length-ratio heuristic flags likely double bonds for terminal (single-heavy-neighbour) N/O atoms and sulfonyl/sulfate sulfur centers - the only cases where bond order actually affects standard-valence hydrogen counting (see "Hydrogen addition" below). This was validated against real PDB ligand structures: a measurably shortened bond (e.g. $\sim 0.85\text{--}0.89\times$ the single-bond reference length for C=O) reliably distinguishes a real carbonyl/imine from an ordinary single bond, while a borderline case (a genuine C-OH at $\sim 0.98\times$) is correctly left alone. This inference does not attempt to detect aromatic-ring unsaturation (uniform bond lengths carry no such signal) - it is deliberately narrow in scope, covering only the well-separated cases that were directly validated.

For Gaussian and ORCA input, this bond-order inference is computed (it's part of the shared connectivity logic) but has no opportunity to matter in practice for hydrogen-counting purposes, since neither reader offers hydrogen addition at all - both formats conventionally specify every atom including hydrogens explicitly as part of a real quantum-chemistry input, and "completing" such a file by guessing hydrogen positions would misrepresent what was actually submitted to those programs. See "Hydrogen addition for H-less input" below.

Preserving response/property data and row alignment

Many real workflows attach a modeled response (e.g. an assay result) or other metadata to each SDF molecule as an ``> <TAG>`` data field, and need that data lined up with RECON's computed descriptors afterward for machine learning.

This instead ``> <TAG>`` fields directly while reading the SDF (``readers/sdf.py``, see the ``data_fields`` dict on each parsed molecule) and carries them through alongside each molecule's descriptors by tracking input position internally. If a molecule fails to parse or fails during descriptor computation, it is dropped from ``recon.csv`` entirely (no blank row, since most ML pipelines treat a half-populated row as noise or a hard error) - its row simply does not appear, and its data fields are dropped with it. Because every surviving row's data fields are read directly from that same molecule's SDF block rather than matched up afterward by name, the response/property columns can never become misaligned with the molecule they belong to, regardless of which molecules earlier in the file were skipped.

This is on by default (``ReconConfig.include_data_fields=True``, CLI ``--no-data-fields`` to disable). The extra columns appear after the descriptor columns, named after their SDF tag (e.g. ``LIC50``, ``EXTREG``).

Included data

This package includes the TAE atom-type database (900+ binary `.dat` files) and a bond-length table — these must be supplied via `--data-dir` (or `ReconConfig.data_dir`).

The bond-length table is expected at `<data-dir>/bond` by default; use `--bond-file` (or `ReconConfig.bond_file`) only if yours lives elsewhere.

Package layout

...

recon6/

fortranio.py	Reader for gfortran unformatted sequential files
taedat.py	Binary TAE .dat record parser
periodic.py	Element symbol / atomic number table
element.py	Atom-name -> element symbol normalizer (ele.f)
ringid.py	Ring-membership detection (ringid.f)
atyper.py	49-character atom-type code generator (atyper)
gettae.py	TAE database index + atom-type matcher (gettae)
bonds.py	Bond-length table loader
descriptors.py	Per-molecule descriptor accumulation (qmf)
sparse.py	Sparse bond-order matrix (memory optimization)
connectivity.py	Shared distance-based connectivity + bond-order inference (PDB, Gaussian, ORCA)
hydrogenate.py	Standard-valence H-addition for H-less SDF/MOL2/PDB input
gnn_export.py	GNN export: mol_to_gnn_dict, GnnJsonWriter, write_mol_json, to_pyg_data
recon.py	Orchestrator: ReconConfig, run_recon
__main__.py	CLI entry point

readers/

sdf.py	SDF (MDL V2000) reader
mol2.py	MOL2 (Tripos) reader
pdb.py	PDB reader (CONNECT or distance-based)
gaussian.py	Gaussian .com/.gif reader (Cartesian coordinates only)
orca.py	ORCA input reader (Cartesian '* xyz' block only)
smiles.py	Simplified SMILES parser

tests/

test_config.py	Shared path config (env-var overridable for portability)
test_readers.py	Unit tests for readers, ele, periodic
test_ringid.py	Ring detection unit tests
test_hydrogenate.py	H-addition geometry and formula-consistency tests
test_pdb_reader.py	PDB element-column, deuterium, bond-order inference tests
test_gaussian_reader.py	Gaussian .com/.gif parsing tests
test_orca_reader.py	ORCA '* xyz' block parsing tests
test_data_fields.py	SDF data-field extraction, CSV quoting, alignment tests
test_streaming.py	Streaming CSV, return_results=False, log-file redirect, error log
test_gnn_export.py	GNN export schema, JSONL/JSON writers, PyG helper
test_integration.py	974-molecule SDF batch vs. Fortran reference
test_integration_toxx.py	278-molecule SDF batch vs. Fortran reference
test_integration_smiles.py	106-line SMILES batch vs. Fortran reference

```

## Running the tests

```bash

```
python -m unittest discover -s tests
```

```

Most of the integration tests depend on sample data (the TAE DATA directory and several large molecule files used during development). Tests whose data isn't present are skipped rather than failed. To point the suite at your own copies of this data, set any of the following environment variables before running (see `tests/test_config.py` for the full list and defaults):

- `RECON6_DATA_DIR``, `RECON6_GAC_SDF``,
- `RECON6_GAC_FF``, `RECON6_TOXX_SDF``, `RECON6_TOXX_FF``,
- `RECON6_TOXX_MOL2``, `RECON6_BENZOX_TXT``, `RECON6_BENZOX_FF``,
- `RECON6_PDB_NNA``, `RECON6_PDB_SV6``.

## Validation

This translation was validated against real Fortran RECON5 output on three independent test sets:

Test set	Molecules	Format	Result
----------	-----------	--------	--------

GAC_withH.sdf	974	SDF	Despite the filename, 971/974 of these molecules have <b>zero</b> explicit hydrogens in the source file, so the Fortran reference for those rows reflects its H-less, unsaturated behavior - not a meaningful comparison now that this release adds missing H's. The 3 molecules that genuinely had explicit H's match the Fortran reference to <0.05%; H-addition for the other 971 is validated by formula consistency (added H count matches each molecule's formula) instead. 3 further molecules are excluded for an unrelated, pre-existing input defect (disconnected halogen atoms) - see Known limitations.
---------------	-----	-----	----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

tox2.sdf	278	SDF	278/278 match to <0.05% (this file is properly H-saturated already)
----------	-----	-----	---------------------------------------------------------------------

Benzoxazines.txt	106 lines	SMILES	94 lines use syntax supported by the original <code>rsmiles.f</code> ; of those, 90+ match Fortran energies exactly
------------------	-----------	--------	---------------------------------------------------------------------------------------------------------------------

Tolerances of <0.05% reflect ordinary floating-point/formatting differences (the Fortran reference output is truncated to ~5-6 significant figures in its text format).

Additional validation beyond the table above: the MOL2 reader was checked against a 278-molecule MOL2 conversion of the same tox2 compound set (reordered relative to the SDF version, so validated by comparing the multiset of computed Energy values rather than row order - 278/278 matched). The PDB reader, hydrogen-addition, and bond-order inference were validated against two real PDB ligand extracts (49 and 103 heavy atoms, no CONECT records, no hydrogens, one with partial deuterium labeling) - added hydrogen counts matched each ligand's molecular formula once carbonyl/sulfonyl bond-order inference was in place. The Gaussian and ORCA readers were cross-checked against each other (identical methane input via both formats



produces identical descriptors) and against the same bond-order-inference logic validated for PDB.

The package has also been run successfully (no crashes) on much larger external datasets, including a 51,449-molecule SDF batch and several other independent SDF datasets supplied during development.

**\*\*Total: 151 tests, all passing\*\*** (116 core pipeline + 14 streaming/memory/log-file/error-log + 21 GNN export).

### Known limitations

SMILES parser: The original ``rsmiles.f`` is a minimal SMILES reader supporting only plain atoms, single/double/triple bonds, branches, and single-digit ring closures — it has no concept of stereochemistry, formal charges, or bond-direction notation. This release matches that scope and explicitly **\*\*rejects\*\*** (rather than silently mis-parsing) SMILES containing:

- Bracket atoms, e.g. ``[C@H]``, ``[N+]``, ``[O-]``
- Bond-direction markers ``/`` and ``\``
- A leading ``(`` before the first atom

A small number of SMILES with reused ring-closure digits in deeply branched systems may also diverge from the Fortran reference; this affects roughly 4 of 94 parseable molecules in the validation set and is a known, low-priority edge case.

Disconnected atoms: If an input structure contains an atom with zero bonds (a data-quality defect — e.g. 3 molecules in the GAC\_withH.sdf test set have chlorine atoms with no bond records at all, despite a nonzero molecular formula), the atom-type matching falls back to an essentially arbitrary TAE database entry. This mirrors the original Fortran's behavior for the same degenerate case (verified by tracing ``atyper``'s logic) rather than being a translation bug, but results for such atoms — and the molecule's aggregate descriptors — should not be trusted. A warning is printed to stderr when this is detected.

Ring sizes: ``ringid`` correctly detects rings of size 3 through 6 (plus several fused-ring codes), but the TAE database (``TAE.LIST``) only contains atom-type entries for 5- and 6-membered rings, so 3- and 4-membered rings have no matching descriptor data in practice.

Bond-order inference does not cover aromatic rings: The bond-length-ratio inference described above (for PDB/Gaussian/ORCA input) only targets terminal N/O double bonds and sulfonyl/sulfate sulfur, which were directly validated. Aromatic ring bonds have essentially uniform length regardless of formal bond order, so there is no geometric signal to detect their unsaturation this way. In practice this only matters for PDB input, since Gaussian and ORCA never get automatic hydrogen addition (see above) - a heavy-atom-only PDB structure with an aromatic ring, run through the H-adder, could miscount hydrogens on that ring for the same reason. If this matters for your workflow, treat it as a follow-up rather than an assumption this release resolves silently.

### Hydrogen addition for H-less input

Many real-world SDF/MOL2 files only specify heavy atoms, and PDB ligand extracts very commonly lack hydrogens entirely. This release detects that condition (``recon6.hydrogenate.needs_hydrogens``, matching the ">10 atoms, 0 H" trigger) and saturates missing valences using standard valence rules (the same default valences RDKit/OpenBabel

apply), with new hydrogen positions placed via local-geometry heuristics (linear / trigonal planar / tetrahedral, depending on each atom's existing neighbor count).

This is on by default for SDF, MOL2, and PDB input; disable it with ``ReconConfig(auto_add_h=False)`` or the CLI's ``--no-add-h`` flag. SMILES input already gets H-saturated by the SMILES parser itself (matching the original ``rsmiles.f``), so the flag has no effect there. For PDB specifically, the same standard-valence formula needs to know real bond order to avoid overcounting H on double-bonded atoms (e.g. a carbonyl oxygen) - see "Distance-based connectivity and bond-order inference" above for how that's handled when the input format itself carries no bond-order information.

Gaussian and ORCA input never get automatic hydrogen addition: Both formats conventionally specify every atom including hydrogens explicitly as part of a real quantum-chemistry input – silently "completing" a heavy-atom-only file by guessing hydrogen positions would misrepresent what was actually submitted to those programs, so this is not offered as an option at all for ``read_gaussian_com`` / ``read_orca_xyz`` (there is no ``auto_add_h`` parameter on either). A heavy-atom-only Gaussian or ORCA file is read and processed exactly as given, with no hydrogens added.

Important caveat - partially-specified hydrogens: The trigger only fires when a molecule has **zero** explicit hydrogens. If a structure has *some* but fewer than its formula implies (for example, a deuterium-labeled compound where only the non-deuterated hydrogens were given explicit positions), no automatic addition happens, and the molecule is processed as-is. This is intentional: a low-but-nonzero H count could reflect deliberate isotope labeling or another legitimate partial specification, and guessing which atoms are "missing" their hydrogens in that case is a different, harder problem than full saturation from zero. If you need this case handled, treat it as a follow-up request rather than an assumption this release makes silently.

When hydrogens are added, a one-line note is printed to stderr per molecule (``Note [name]: added N hydrogen(s) ...``) so it's visible in batch logs.

## Authors

Several authors have put in a LOT of work developing the original package over the years (in alphabetical order): Curt Breneman, William P. Katt, Martin Martinov, Marlon Rhem, Dominic Ryan, N. Sukumar, Tracy R. Thompson, Christopher Whitehead, Dechuan Zhuang.

## References

1. N. Sukumar and Curt M. Breneman, "QTAIM in Drug Discovery and Protein Modeling" in "The Quantum Theory of Atoms in Molecules: From Solid State to DNA and Drug Design" C.F. Matta & R.J. Boyd, Eds. (Wiley-VCH, 2007) ISBN: 9783527307487
2. Christopher E. Whitehead, Curt M. Breneman, N. Sukumar and M. D. Ryan, "Transferable Atom Equivalent Multi-Centered Multipole Expansion Method" *J. Comp. Chem.* **24**, 512-529 (2003) DOI: 10.1002/jcc.10240
3. Breneman, C.M., et al., *Electron Density Modeling of Large Systems Using the Transferable Atom Equivalent Method*. Computers & Chemistry, 1995. **19**(3): p.161.
4. Thompson, T.R., *Construction of a Library of Transferable Atom Equivalents*, in *Chemistry*. 1994, Rensselaer Polytechnic Institute: Troy.
5. Bader, R.F.W., *Atoms in Molecules: A Quantum Theory*. 1990, Oxford: Oxford Univ. Press.